

第7回: 履歴の永続化 (SQLite)

LLMアプリケーション基礎

今日のゴール

会話履歴を **SQLite** に保存 して、リロード・再起動しても消えない状態を作る

今日の流れ

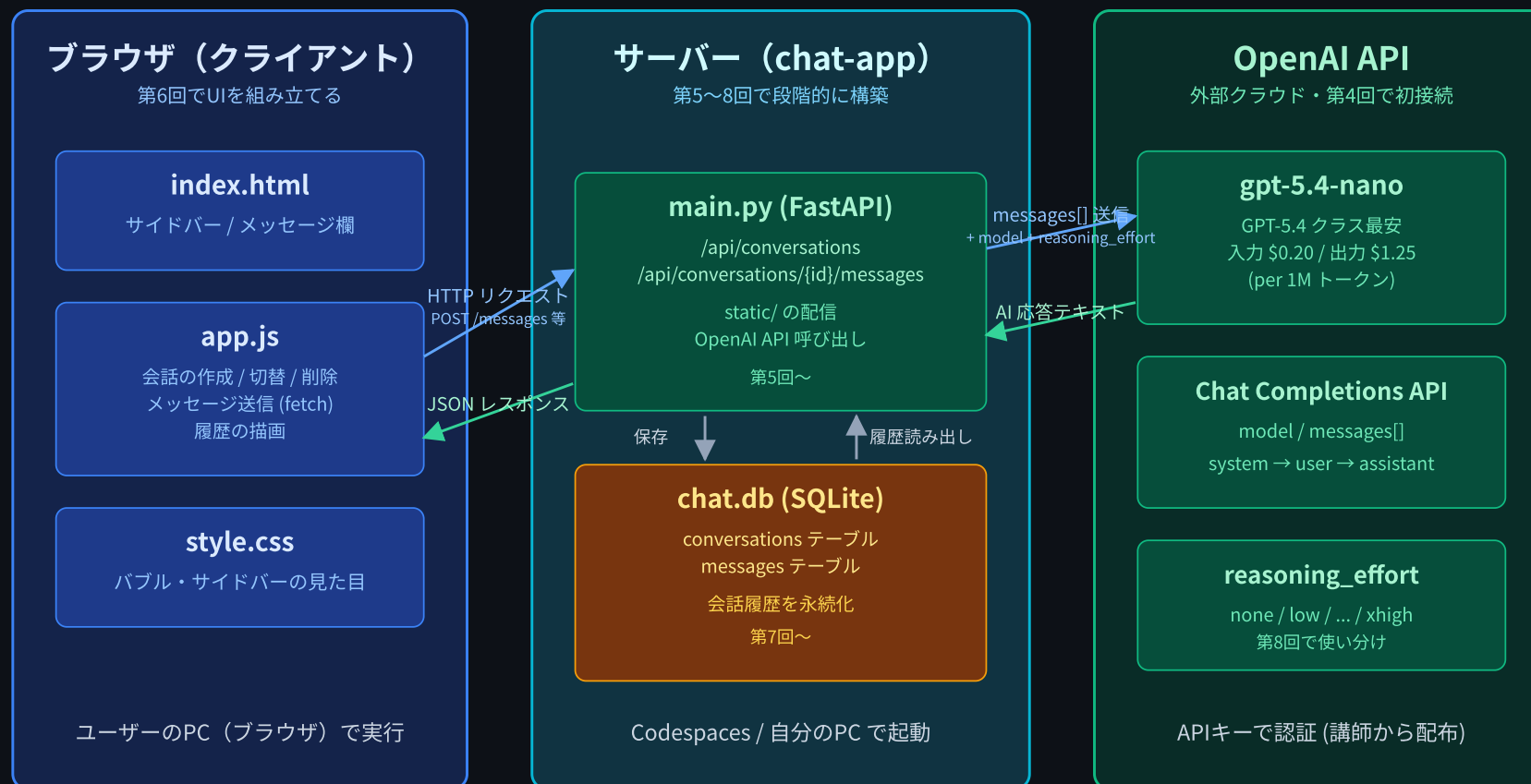
前半

- なぜ永続化が必要か（フロント保持の限界）
- スキーマ設計 と `init_db()` でテーブル作成
- 各関数で `connect` → `close` する書き方

後半

- 保存 / 読み込み / API へ流す処理を実装
- フロント側の差分（履歴をサーバから取得）
- 演習: ブラウザリロード・サーバ再起動しても会話が残ることを確認

chat-app の全体像



今日は中央に **SQLite (chat.db)** を足し、履歴をリロードしても消えないようにする

第6回の振り返り と 課題

第6回で作ったものはこうだった

- フロントの JS が `messages` 配列を メモリ上 に持つ
- 送信のたびにその配列を丸ごとサーバに POST
- サーバはステートレス: 受け取った履歴をそのまま OpenAI に転送

問題: ブラウザをリロードしたら JS 変数は空に戻る

- 会話を続けるには「タブを閉じない」「リロードしない」が条件
- 普通のチャットアプリとしてあり得ない

どこに保存すれば消えないか

場所	消える？	問題点
JS のメモリ変数	リロードで消える	これが第6回の状態
<code>localStorage</code>	残る	そのブラウザにしか残らない / バックアップ困難
サーバの SQLite	残る	別端末からでも見られる / バックアップしやすい

→ 今日は サーバ側 SQLite に置く

アーキテクチャの変化

アーキテクチャの変化: 履歴の置き場が「ブラウザ」から「サーバ」へ

第6回まで: フロントが履歴を持つ

第7回~: サーバが履歴を持つ



サーバが履歴を持つメリットと道具立て

メリット

- リロード・再起動で消えない / 別端末でも見られる(原理上)
- フロントは「画面の表示」に集中できる(状態管理がシンプル)
- 第8回で複数会話に拡張するときの土台になる

道具: SQLite (第3回でおさらいした軽量DB)

- ファイル1つ (`chat.db`) で完結。サーバ不要
- Python 標準ライブラリの `sqlite3` モジュール → 追加インストール不要

DB スキーマ設計

今回は テーブル1つだけ にする

messages テーブル

カラム	型	説明
id	INTEGER PRIMARY KEY AUTOINCREMENT	一意な番号 (自動採番)
role	TEXT	"user" または "assistant"
content	TEXT	発言の本文
created_at	TEXT (DEFAULT CURRENT_TIMESTAMP)	作成時刻

CREATE TABLE 文

```
CREATE TABLE IF NOT EXISTS messages (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  role        TEXT NOT NULL,  
  content     TEXT NOT NULL,  
  created_at  TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

- `IF NOT EXISTS` で「既にあれば作らない」 (= 起動のたびに安全に呼べる)
- `created_at` は省略時に **DBが現在時刻を自動で入れてくれる**
- 第8回で `conversations` テーブルを追加して `conversation_id` を生やす

なぜ system プロンプトはDBに入れない？

- 今回は単一会話・固定の振る舞いなので サーバ側の定数 で十分
- DB に入れるのは「動的に変えたいもの」だけにする (YAGNI)
- 第8回で「会話ごとに違う system プロンプト」をやるときに `conversations.system_prompt` カラムを追加する

```
DEFAULT_SYSTEM_PROMPT = (  
    "あなたは親切で丁寧なアシスタントです。日本語で回答してください。"  
)
```

`init_db()`: テーブル作成

```
import sqlite3

DATABASE = "chat.db"

def init_db():
    """データベースとテーブルを初期化する"""
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS messages (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            role TEXT NOT NULL,
            content TEXT NOT NULL,
            created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    conn.close()

# アプリ起動時に1回呼ぶ
init_db()
```

接続は使うたびに `connect` → `close`

各 API ハンドラの中で毎回 `sqlite3.connect` して、最後に `close` する

```
def get_something():  
    conn = sqlite3.connect(DATABASE)  
    # 結果を辞書のように row["role"] で取り出せるようにする  
    conn.row_factory = sqlite3.Row  
    cursor = conn.cursor()  
  
    cursor.execute("SELECT ...")  
    rows = cursor.fetchall()  
  
    conn.close()  
    return rows
```

- 関数ごとに自分で開いて、自分で閉じる
- `return` の前に `conn.close()` を書くだけ
- 「閉じる場所」がコードを読めば一目で分かるのがメリット

API 設計 (今回はシンプル)

メソッド	パス	役割
GET	<code>/api/messages</code>	全メッセージを古い順に返す
POST	<code>/api/messages</code>	ユーザー発言を受け取り、AI返答を返す

- まだ会話は1つだけなので URL に `conversation_id` は無い
- 第8回で `/api/conversations/{id}/messages` へ拡張する

GET /api/messages の実装

```
@app.get("/api/messages")
def get_messages():
    """全メッセージを古い順で返す"""
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    cursor.execute("""
        SELECT id, role, content, created_at
        FROM messages
        ORDER BY id
    """)
    rows = cursor.fetchall()

    conn.close()
    return [
        {"id": r["id"], "role": r["role"],
         "content": r["content"], "created_at": r["created_at"]}
        for r in rows
    ]
```

POST `/api/messages` の処理の流れ

1. リクエストから user のメッセージ本文を取り出す
2. **DB に user メッセージを保存** (これで会話履歴の一部になる)
3. DB から **過去メッセージ全件** を古い順に取り出す
4. 先頭に system を付けて OpenAI へ送る
5. AI の返答を **DB に保存**
6. 返答をフロントへ返す

POST /api/messages 実装 (1/2)

```
@app.post("/api/messages", status_code=201)
def send_message(user_message: MessageCreate):
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    # 1. ユーザーメッセージをDBに保存
    cursor.execute(
        "INSERT INTO messages (role, content) VALUES (?, ?)",
        ("user", user_message.content),
    )
    conn.commit()

    # 2. 過去メッセージ全件を古い順に取り出す
    cursor.execute(
        "SELECT role, content FROM messages ORDER BY id"
    )
    past_rows = cursor.fetchall()
```

POST /api/messages 実装 (2/2)

```
# 3. system + 過去全件 を OpenAI 形式に組み立て
messages_for_api = [
    {"role": "system", "content": DEFAULT_SYSTEM_PROMPT},
]
for row in past_rows:
    messages_for_api.append(
        {"role": row["role"], "content": row["content"]}
    )

# 4. OpenAI 呼び出し
response = client.chat.completions.create(
    model=MODEL_NAME,
    messages=messages_for_api,
    reasoning_effort=REASONING_EFFORT,
)
assistant_content = response.choices[0].message.content

# 5. AI返答をDBに保存
cursor.execute(
    "INSERT INTO messages (role, content) VALUES (?, ?)",
    ("assistant", assistant_content),
)
conn.commit()

conn.close()
return {"role": "assistant", "content": assistant_content}
```

フロント側の変化 (1)

第6回からの差分はとても少ない

- `messages` 配列の保持を **やめる** (サーバが持つ)
- ページロード時に `GET /api/messages` で過去全件を取得して描画
- 送信時は `POST /api/messages` を呼んで返ってきた1件を画面に追加するだけ

フロント側の変化 (2): ページロード時

```
// ページが読み込まれたら、まず過去のメッセージを取得して描画
async function loadMessages() {
  const response = await fetch("/api/messages");
  const messages = await response.json();
  messages.forEach((m) => appendMessage(m.role, m.content));
}

loadMessages();
```

- 第6回では「空のメッセージリストから始まる」だったのが
- 今回は「DBに保存されている履歴から始まる」になる
- これだけで「リロードしても消えない」が実現する

フロント側の変化 (3): 送信時

```
async function sendMessage() {
  const content = input.value.trim();
  appendMessage("user", content); // 自分の発言は即表示

  const res = await fetch("/api/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ content }), // 履歴を送らない!
  });
  const assistant = await res.json();
  appendMessage("assistant", assistant.content);
}
```

- リクエストボディは **今回の1発言だけ**
- 過去履歴はサーバが DB から自分で取り出す

DB の中身を覗いてみる

サーバを動かしながら別ターミナルで:

```
sqlite3 chat.db "SELECT id, role, substr(content,1,40), created_at FROM messages;"
```

- `chat.db` は `python main.py` 初回起動時に自動で作られる
- このファイルを消せば履歴は全部リセットされる
- `.gitignore` に `*.db` を入れる (履歴をコミットしない)

ハマりどころ

- `conn.commit()` を忘れる → 書き込んだつもりが反映されない
- `?` プレースホルダを使わず文字列連結 → SQL インジェクション
- `init_db()` を呼び忘れる → 起動時に `no such table: messages`
- `row_factory` を設定し忘れ → `row["role"]` ではなく `row[0]` を使う羽目になる

演習

`session07/exercise/` をコピーするか、自分の第6回コードを改造して:

1. `messages` テーブルを持つ SQLite を初期化
2. ブラウザで何往復か会話する
3. タブをリロード → 過去の会話が見えるか確認
4. サーバを `ctrl+c` で止めて再起動 → 過去の会話がまだあるか確認
5. `sqlite3 chat.db "SELECT * FROM messages;"` で DB の中身を確認

余裕があれば

- `chat.db` を削除して起動 → まっさらから始まることを確認
- DevTools の Network タブで POST のリクエストボディが「1件だけ」になっていることを確認

本日のまとめ

学んだこと

1. フロントが履歴を持つ世界から、サーバが履歴を持つ 世界へ移行した
2. `messages` テーブル1つで十分実用になる
3. `init_db()` と 各関数での `connect / close` パターンは第8回でもそのまま使う
4. フロントの責務が減って、UIに集中できるようになった
5. リロード・サーバ再起動でも会話が残るようになった

次回予告

第8回: 複数会話の切り替え + 仕上げ

`conversations` テーブルを追加し、`messages` に `conversation_id` 外部キーを生やす。サイドバーから会話を切り替え・新規作成・削除できるようにして、ついに **chat-app 完成**。最後に発展テーマ（Tool use / RAG / エージェント）の概念も紹介する。

提出物

実習で作成したファイルをフォームから提出してください:

1. `session07/exercise/` の chat-app (SQLite永続化版) の GitHub のURL
 - 例: `https://github.com/ユーザー名/リポジトリ名/tree/main/session07/exercise`

お疲れ様でした！